Registro de Decisão Arquitetural

Descomplica Sports

Versão 1.0 | 05/11/2026 | Marco 3
Projeto e Engenharia de Software | A3 | 2026.2

Responsável: Kauan Martins Silva (RA 8261106153).

Autores: Guilherme Eduardo Silva (RA 826177287); Gustavo de Souza (RA 8261108661); Cássio Martins Ceribelli (RA 8261102908); Ryan Guilherme Almeida de Camargo (RA 826195315); Kauan Martins Silva (RA 8261106153); Ícaro Henrique Timóteo Severino (RA 8261100640).

Base: Documento de Requisitos v1.0 (01/10/2026), Casos de Uso e Diagramas de Classes do Marco 2 e diagramas de sequência dos UC-08 e UC-09. Os identificadores RF, RN e RNF seguem essa numeração. A documentação dos serviços externos citados foi consultada em 03/11/2026.

0 Índice de ADRs e diagrama da arquitetura

Cada ADR é imutável. Se uma decisão mudar, o ADR original não é reescrito: cria-se um novo ADR com status "substitui o ADR-00X", preservando o histórico do raciocínio.

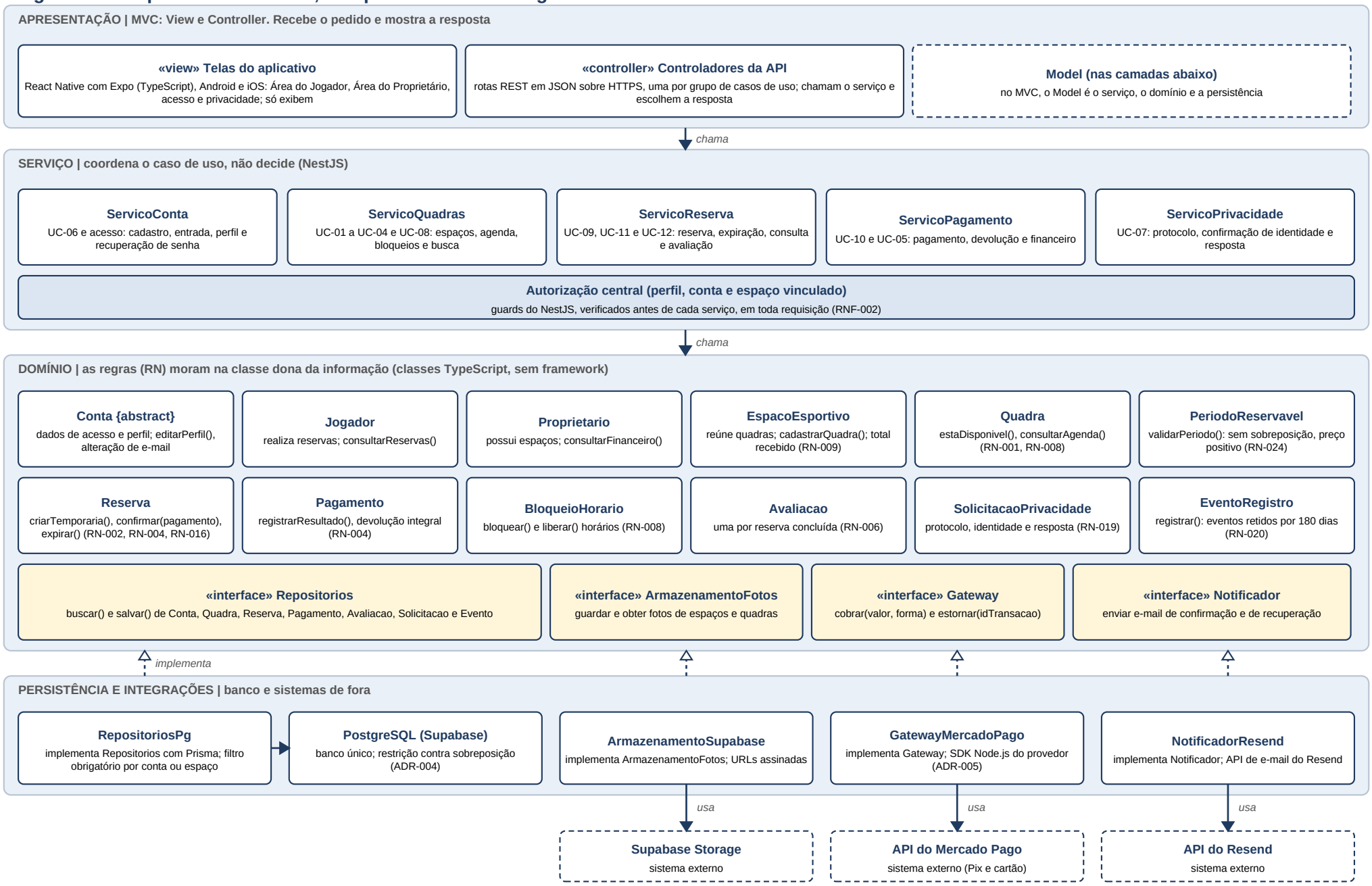
ADR	Título (a decisão)	Status	Data
ADR-001	Adotar arquitetura em quatro camadas, com MVC na apresentação	aceita	05/11/2026
ADR-002	Usar TypeScript no aplicativo (React Native com Expo) e na API (Node.js com NestJS)	aceita	05/11/2026
ADR-003	Expor a API como REST em JSON sobre HTTPS, com token de acesso curto e token de renovação revogável	aceita	05/11/2026
ADR-004	Usar PostgreSQL gerenciado (Supabase) como banco único e impedir sobreposição de horários por restrição no banco	aceita	05/11/2026
ADR-005	Adotar o Mercado Pago como provedor de pagamento, atrás da interface Gateway	aceita	05/11/2026
ADR-006	Definir a expiração da reserva temporária pelo relógio do servidor, com o prazo gravado na Reserva	aceita	05/11/2026
ADR-007	Centralizar a autorização por perfil e vínculo de espaço antes de cada serviço	aceita	05/11/2026
ADR-008	Usar serviços gerenciados para hospedagem (Render), e-mail (Resend), fotos (Supabase Storage) e cópias de segurança próprias a cada 6 horas	aceita	05/11/2026

Resumo das decisões tecnológicas

Item	Escolha	ADR
Padrão arquitetural	Quatro camadas (apresentação com MVC, serviço, domínio, persistência e integrações), monólito modular	ADR-001
Linguagem	TypeScript no aplicativo e na API	ADR-002
Aplicativo móvel	React Native com Expo, um aplicativo para Android e iOS	ADR-002
API	Node.js com NestJS, em módulos com injeção de dependências	ADR-002
Estilo da API	REST em JSON sobre HTTPS, documentada em OpenAPI	ADR-003
Autenticação	Senha com Argon2id, token de acesso curto e token de renovação revogável	ADR-003
Banco de dados	PostgreSQL gerenciado (Supabase), acesso pelo Prisma, restrição de exclusão contra sobreposição	ADR-004

Item	Escolha	ADR
Pagamento	Mercado Pago (Pix e cartão), SDK para Node.js, notificações por webhook, atrás da interface Gateway	ADR-005
Prazo da reserva	Expiração gravada na Reserva pelo relógio do servidor	ADR-006
Autorização	Guards centrais do NestJS e filtro obrigatório por conta ou espaço nos repositórios	ADR-007
Hospedagem da API	Render (serviço web e job agendado)	ADR-008
E-mail	API do Resend	ADR-008
Fotos	Supabase Storage, com URLs assinadas	ADR-008
Cópias de segurança	pg_dump a cada 6 horas, criptografado, em armazenamento separado, retido por 30 dias	ADR-008
APIs externas usadas	Mercado Pago (pagamentos, devoluções e notificações), Resend (e-mail) e Supabase (banco e fotos). Sem GPS nem mapas (RN-001)	ADR-005 e ADR-008

Diagrama da arquitetura: camadas, componentes e tecnologias



Leitura do diagrama

- **Direção das dependências:** cada camada só chama a de baixo. A seta de «implementa» sobe porque o domínio define o contrato e a persistência o cumpre. De baixo para cima, ninguém chama.
- **MVC:** View e Controller moram na apresentação. O Model é tudo o que está do serviço para baixo.
- **Sistemas externos** (Mercado Pago, Resend e Supabase Storage) só aparecem atrás de uma «interface» definida pelo domínio. Nos testes, entram versões falsas no lugar.
- **Conferência no Pull Request:** a revisão olha os imports. A apresentação não importa a persistência, e o domínio não importa nada das camadas de cima nem o driver do banco.

O que veio do Marco 2 e o que é proposta

Componente	Origem	Situação
12 classes de domínio (Conta, Jogador, Proprietario, EspacoEsportivo, Quadra, PeriodoReservavel, Reserva, Pagamento, BloqueioHorario, Avaliacao, SolicitacaoPrivacidade, EventoRegistro)	Diagrama de classes do Marco 2 (UC-01 a UC-12)	Do Marco 2, sem alteração
Gateway (cobrar e estornar)	Sequência do UC-09: cobrança e estorno da devolução integral	Do Marco 2; formalizado como interface
Serviços (5)	Objeto "Sistema" das sequências dos UC-08 e UC-09, dividido por grupo de casos de uso	Proposta do ADR-001
Telas e controladores	«boundary» Aplicativo da sequência do UC-08	Proposta do ADR-001
Repositorios, ArmazenamentoFotos e Notificador	Necessidade de gravar dados, guardar fotos (RF-007) e enviar e-mails (RF-012, RF-016)	Proposta do ADR-001
Autorização central	RNF-002 e RN-005	Proposta do ADR-007

Teste das 72 horas: RN-002 (prazo de 10 minutos da reserva temporária)

A RN-002 é a regra que o cliente mais provavelmente vai ajustar, por exemplo para 15 minutos. Seguindo o UC-09: o passo 3 (criação da reserva, "expira em 10 minutos") acontece em **Reserva.criarTemporaria()**, no domínio, que grava expiraEm. O ServicoReserva apenas chama; o aplicativo exibe o contador a partir de expiraEm sem conhecer o 10; o PostgreSQL guarda expiraEm, mas não calcula nada. A mensagem 10 ("tempo expirou") chama Reserva.expirar(), que compara com expiraEm. A regra aparece em **uma classe e uma camada**. A RN-002 também usa 10 minutos como antecedência mínima, e a RN-014 usa 10 minutos para parar novas reservas antes da manutenção: são regras diferentes e devem ser constantes diferentes no domínio.

1 Identificação do ADR

ADR-001 · Adotar arquitetura em quatro camadas, com MVC na apresentação

Status: aceita | **Data:** 05/11/2026

2 Contexto

Jogadores e proprietários usam o mesmo aplicativo móvel, e as regras RN-002, RN-003, RN-004 e RN-005 precisam valer da mesma forma em qualquer tela e em qualquer canal que surja depois. Nos diagramas de sequência do Marco 2 (UC-08 e UC-09), um único objeto "Sistema" faz tudo: confere disponibilidade, cria a reserva, chama o gateway e confirma. As classes do diagrama de classes ainda não têm endereço.

O requisito que mais pesa é o RNF-002: nenhuma conta pode alcançar reservas, agenda ou financeiro de outra, e dados de cartão ficam somente com o provedor (RN-012). Também estão em jogo a busca em até 3 segundos (RNF-001) e os registros de auditoria por 180 dias (RNF-007).

3 Decisão

Adotamos quatro camadas: apresentação (MVC), serviço, domínio e persistência e integrações. Cada camada só chama a de baixo.

A apresentação contém as telas do aplicativo (View) e os controladores da API (Controller). O serviço tem um objeto de controle por grupo de casos de uso (ServicoConta, ServicoQuadras, ServicoReserva, ServicoPagamento e ServicoPrivacidade), que coordena e não decide. O domínio reúne as classes do diagrama de classes (Reserva, Quadra, Pagamento e as demais), onde moram as regras, e define as interfaces Repositorios, ArmazenamentoFotos, Gateway e Notificador. A persistência e as integrações cumprem essas interfaces: o banco, o provedor de pagamento, o armazenamento de fotos e o envio de e-mail só aparecem atrás delas. A aplicação é implantada como um único serviço (monólito modular).

4 Alternativas consideradas

MVC com regra no controlador
O que era: As regras ficariam nas rotas da API, ao lado do acesso ao banco. **Por que não:** A RN-004 (aprovação antes da expiração) e a RN-005 (dados visíveis por perfil) seriam repetidas em cada rota que as usa, e uma mudança de prazo exigiria abrir vários arquivos.

Três camadas sem domínio (regra no serviço)
O que era: O serviço decidiria, e as classes seriam apenas dados com get e set. **Por que não:** O serviço engordaria e as classes virariam modelo anêmico, repetindo o problema do controlador uma camada abaixo. As sequências do Marco 2 já atribuem comportamento às classes, como confirmar(pag).

Microsserviços (quadras, reservas e pagamentos)
O que era: Cada área seria uma aplicação separada. **Por que não:** Descartado agora: a equipe tem seis pessoas, há um banco só, e nenhum RNF pede escalar uma parte isolada. Reserva e pagamento precisam de consistência entre si (RN-003 e RN-004), e separá-los exigiria coordenar transações entre aplicações.

5 Consequências

Positivas
- Cada regra de negócio mora em uma classe do domínio, em um lugar só.

- Trocar o provedor de pagamento, o banco ou o serviço de e-mail altera apenas a camada de persistência e integrações.
- O domínio e os serviços podem ser testados sem banco e sem provedor, com um GatewayFalso.

Negativas

- Mais arquivos: operações simples, como cadastrar uma quadra, atravessam as quatro camadas.
- Nada impede um import proibido além da revisão do Pull Request.
- Cada camada é mais uma chamada na busca, o que pesa sobre o RNF-001.
- Exige disciplina para manter as regras no domínio e não no serviço.

6 Requisitos afetados

- **Atende:** RNF-002, RNF-007
- **Compromete:** RNF-001 (medir a busca nas 20 consultas cronometradas, pois cada camada acrescenta uma chamada)

7 Riscos e gatilho de reavaliação

Reavaliar se a busca passar de 3 segundos mesmo com índices, se o aplicativo ganhar equipe própria ou se surgir um novo canal, como API para parceiros, ou o bate-papo (RN-010).

1 Identificação do ADR

ADR-002 · Usar TypeScript no aplicativo (React Native com Expo) e na API (Node.js com NestJS)

Status: proposta | **Data:** 05/11/2026

2 Contexto

A RN-015 define a primeira versão para celulares Android e iOS, deixando o desktop para depois. A equipe tem seis estudantes e um semestre, e o aplicativo e a API trocam os mesmos dados (reservas, pagamentos, períodos). O RNF-005 exige interface sem corte em 360 e 390 pixels nas duas plataformas, e o RNF-009 exige contraste, ampliação a 200% e leitura por leitor de tela.

A organização em camadas do ADR-001 também pede um framework que facilite separar módulos e trocar implementações por interfaces, para testar as regras sem banco e sem provedor.

3 Decisão

Escrevemos o sistema inteiro em TypeScript: o aplicativo com React Native e Expo, e a API com Node.js e NestJS.

Um único aplicativo atende Android e iOS e reúne os perfis de jogador e de proprietário. A API usa NestJS, cujos módulos e injeção de dependências sustentam as camadas do ADR-001: controladores na apresentação, serviços na camada de serviço, e implementações das interfaces na persistência. As classes do domínio são TypeScript simples, sem dependência do framework. Os testes automatizados usam Jest, e os tipos dos dados trocados entre aplicativo e API podem ser compartilhados.

4 Alternativas consideradas

Aplicativos nativos (Kotlin e Swift) com API em Java e Spring Boot
O que era: Duas bases de código de aplicativo e uma API em Java. **Por que não:** Dobraria o trabalho de interface e exigiria três linguagens para uma equipe de seis em um semestre, sem que algum RNF exija desempenho nativo.

Flutter (Dart) no aplicativo
O que era: Um aplicativo único em Dart, com a API em outra linguagem. **Por que não:** Acrescenta uma linguagem a mais sem ganho para os RNF. Manter uma só linguagem reduz o custo de aprender e de revisar código.

React Native no aplicativo e API em Java com Spring Boot
O que era: Duas linguagens, com Java alinhado à orientação a objetos do curso. **Por que não:** Exigiria dois ecossistemas, dois conjuntos de ferramentas e a duplicação dos tipos de dados, e nenhum RNF exige a plataforma Java.

5 Consequências

Positivas

- Uma linguagem só em todo o sistema, o que simplifica a revisão dos Pull Requests.
- Um aplicativo atende Android e iOS, e os RNF-005 e RNF-009 são verificados sobre uma base de código.
- Os módulos e a injeção de dependências do NestJS combinam com as interfaces do domínio.

Negativas

- Node.js processa em uma linha de execução principal: trabalho pesado ou consulta lenta atrasa outras requisições.
- O React Native depende de bibliotecas de terceiros e de atualizações de plataforma.

- Os tipos do TypeScript não existem em execução, então a entrada da API precisa de validação explícita.
- O NestJS tem curva de aprendizado (decoradores e módulos) para quem não o conhece.

6 Requisitos afetados

- **Atende:** RNF-005, RNF-009 (verificáveis nas duas plataformas com uma base de código)
- **Compromete:** RNF-001 (uma consulta lenta ocupa a execução principal do Node.js; medir nas 20 consultas cronometradas)

7 Riscos e gatilho de reavaliação

Reavaliar se o aplicativo não cumprir o RNF-005 ou o RNF-009 em Android ou iOS por limitação do framework, ou se a versão desktop (RN-015) for criada; nesse caso a API segue igual e só a apresentação muda.

1 Identificação do ADR

ADR-003 · Expor a API como REST em JSON sobre HTTPS, com token de acesso curto e token de renovação revogável

Status: proposta | **Data:** 05/11/2026

2 Contexto

A RN-012 exige transmissão criptografada e senhas nunca em texto puro. A RN-018 determina que a entrada use e-mail confirmado e senha sem revelar qual credencial falhou, que sair encerre a sessão e que a troca de senha invalide a senha anterior, os outros links de recuperação e as outras sessões. O link de recuperação é de uso único e vale 15 minutos.

O aplicativo móvel precisa manter a pessoa conectada entre aberturas, e o RNF-002 exige que cada requisição saiba quem é a conta e qual é o perfil.

3 Decisão

A API é REST, em JSON, sobre HTTPS e documentada em OpenAPI, e a sessão é um token de acesso de curta duração mais um token de renovação revogável.

As rotas seguem os recursos do sistema: conta, quadras, agenda, reservas, pagamentos, financeiro e privacidade, mais uma rota própria para as notificações do provedor de pagamento (ADR-005). As senhas são guardadas com hash Argon2id. O token de acesso dura poucos minutos e leva o perfil e a conta. O token de renovação é aleatório, rotativo e guardado no banco apenas como hash, em uma tabela de sessões: sair o revoga, e a troca de senha revoga todos. Os links de confirmação de e-mail e de recuperação são tokens aleatórios de uso único, também guardados como hash, com validade (15 minutos na recuperação). A entrada e a recuperação limitam tentativas e respondem de forma genérica.

4 Alternativas consideradas

Sessão por cookie mantida no servidor
O que era: A API guardaria a sessão e enviaria um cookie ao cliente. **Por que não:** Cookies funcionam melhor em navegador. No aplicativo nativo exigiriam gerenciar o cookie manualmente e um armazenamento de sessão compartilhado.

GraphQL
O que era: Um único ponto de consulta, com o cliente escolhendo os campos. **Por que não:** Os 12 casos de uso são poucos e bem definidos, a flexibilidade de consulta não é exigida, e a autorização por campo complicaria o RNF-002 e a RN-005.

Token JWT de longa duração, sem revogação
O que era: Um único token valendo por dias ou semanas. **Por que não:** A RN-018 exige invalidar sessões ao sair e ao trocar a senha, o que um token sem revogação não permite.

5 Consequências

Positivas

- As sessões são revogáveis, o que cumpre a RN-018.
- O contrato da API fica documentado em OpenAPI e é simples de testar.
- Senhas, tokens de renovação e links ficam guardados só como hash.

Negativas

- A renovação do token consulta o banco, o que acrescenta uma leitura por renovação.
- Depois de revogado, o token de acesso ainda vale até expirar, por isso sua duração é curta.
- Há mais código de sessão e de limitação de tentativas para escrever e testar.

6 Requisitos afetados

- **Atende:** RNF-002, RNF-007 (entradas, saídas e trocas de senha podem ser registradas)
- **Compromete:** RNF-001 (a verificação do token em cada requisição tem custo pequeno, a medir na busca)

7 Riscos e gatilho de reavaliação

Reavaliar se surgir uma API aberta para parceiros ou entrada por conta de terceiros, ou se o prazo de segurança da RN-018 mudar.

1 Identificação do ADR

ADR-004 · Usar PostgreSQL gerenciado (Supabase) como banco único e impedir sobreposição de horários por restrição no banco

Status: proposta | **Data:** 05/11/2026

2 Contexto

A RN-003 determina que, para períodos sobrepostos da mesma quadra, prevalece a primeira solicitação aceita em operação indivisível, e que a segunda é recusada antes do pagamento. A RN-001 e a RN-008 exigem que reservas temporárias, confirmadas e bloqueios sejam considerados juntos na disponibilidade.

Quadra.estaDisponivel() implementa a regra, mas dois jogadores podem perguntar antes de qualquer um gravar e ambos passarem. Os registros também precisam permanecer íntegros (RNF-007) e ser recuperáveis com perda máxima de 6 horas (RNF-008).

3 Decisão

Adotamos um único banco PostgreSQL, gerenciado pelo Supabase, como fonte de verdade: a regra de disponibilidade fica no domínio e a garantia contra conflito fica no banco.

A regra mora em Quadra.estaDisponivel(inicio, termino). A garantia mora na persistência: reservas ativas (temporárias e confirmadas) e bloqueios ativos gravam uma ocupação de horário em uma única tabela, com restrição de exclusão sobre quadra e intervalo de tempo, válida só enquanto a ocupação está ativa. A restrição combina igualdade de quadra com sobreposição de intervalo e usa a extensão btree_gist. A criação da reserva ocorre em uma transação: a segunda inserção para o mesmo período falha, e o repositório a traduz em indisponibilidade. O acesso ao banco usa o Prisma, e a restrição é escrita à mão em uma migração SQL, porque o Prisma não a modela. O Supabase é usado como banco gerenciado: o aplicativo nunca o acessa diretamente, somente a API.

4 Alternativas consideradas

Verificar e gravar apenas no domínio ou no serviço
O que era: A API consultaria estaDisponivel() e, se livre, gravaria a reserva, sem restrição no banco. **Por que não:** Há uma janela de concorrência entre a consulta e a gravação, e a correção dependeria de todo caminho de código lembrar de fazer a verificação.

Banco NoSQL de documentos
O que era: Reservas e quadras seriam documentos independentes. **Por que não:** Os dados são fortemente relacionados (quadra, período, reserva, pagamento) e o total do financeiro (RN-009) é uma agregação natural em SQL. Perderíamos a restrição de sobreposição nativa.

Aplicativo acessando o banco direto pela API automática do Supabase
O que era: O aplicativo leria e gravaria no banco, protegido por regras de linha. **Por que não:** Contraria o ADR-001: as regras ficariam no cliente, e regras que dependem do estado da reserva (RN-005) ficariam difíceis de testar.

5 Consequências

Positivas

- Nem um erro de programação consegue gravar duas ocupações sobrepostas.
- Um único conjunto de dados para copiar, restaurar e reconciliar (RNF-008).
- Consultas de agenda e de financeiro são simples e rastreáveis em SQL.

Negativas

- A restrição de exclusão é recurso do PostgreSQL e exige uma migração escrita à mão.

- Como a restrição não atravessa tabelas, reservas e bloqueios precisam compartilhar a tabela de ocupações.
- O banco único é ponto único de falha e cresce por ampliação vertical.
- Horários muito disputados geram contenção e exigem transações curtas.

6 Requisitos afetados

- **Atende:** RNF-007, RNF-008, RNF-002 (dados centralizados e filtráveis por conta)
- **Compromete:** RNF-001 (a busca por data e horário depende de índices por cidade, bairro e período)

7 Riscos e gatilho de reavaliação

Reavaliar se os índices não bastarem para o RNF-001 ou se a plataforma passar a operar em várias regiões com bancos separados.

1 Identificação do ADR

ADR-005 · Adotar o Mercado Pago como provedor de pagamento, atrás da interface Gateway

Status: proposta | **Data:** 05/11/2026

2 Contexto

O RF-004 exige pagamento por Pix ou cartão de crédito. A RN-012 determina que dados completos de cartão e códigos de segurança sejam tratados pelo provedor, sem armazenamento na plataforma. A RN-004 estabelece que somente a aprovação recebida antes da expiração confirma a reserva, que a aprovação tardia gera devolução integral e que repetições não duplicam cobrança nem reserva.

O resultado de um pagamento pode chegar na resposta da cobrança ou depois, especialmente no Pix, e pode ser entregue mais de uma vez (UC-10, fluxo 4c). É preciso escolher o provedor e definir o que prova que um pagamento foi aprovado. A escolha precisa de SDK para Node.js, ambiente de testes e devolução total por API.

3 Decisão

Adotamos o Mercado Pago como provedor de pagamentos (Pix e cartão de crédito), integrado pela API e pelo SDK oficial para Node.js, e o isolamos atrás da interface Gateway.

O GatewayMercadoPago implementa Gateway.cobrar(valor, forma) e Gateway.estornar(idTransacao) e é o único componente que conhece o provedor. No cartão, o aplicativo obtém um token do cartão diretamente com o provedor e a API recebe somente o token. No Pix, a API cria o pagamento e devolve ao aplicativo o código para pagar. O resultado chega pela resposta ou pela notificação (webhook) do provedor; a rota valida a assinatura e consulta o pagamento na API do provedor antes de registrá-lo. Pagamento.registrarResultado() usa o identificador da transação como chave única e ignora repetições. Em seguida, Reserva.confirmar(pagamento) decide pelo prazo (ADR-006). Se a aprovação for tardia, a reserva continua expirada e o serviço chama Gateway.estornar, que pede a devolução total, deixando a situação como solicitada até o provedor confirmar. O token de acesso do provedor fica em variável de ambiente, nunca no aplicativo.

A validade do Pix no provedor não coincide necessariamente com os 10 minutos da reserva, então a aprovação tardia é um caso normal e é o que o fluxo de devolução cobre. Desenvolvimento e testes usam as credenciais de teste do provedor e um GatewayFalso.

4 Alternativas consideradas

Checkout Pro, com redirecionamento ao site do provedor
O que era: O jogador sairia do aplicativo para pagar na página do provedor. **Por que não:** Dificulta manter o contador e a situação da reserva visíveis (RN-002 e RN-022) e perde o controle do fluxo do prazo.

Pix direto pela API de um banco, sem gateway
O que era: A plataforma se integraria a um banco para gerar e conferir cobranças Pix. **Por que não:** Não cobre o cartão de crédito, e relatos de desenvolvedores indicam que o Pix gerado em sandbox de bancos não é pagável, restando testar em produção.

Outro gateway nacional (PagBank, Pagar.me)
O que era: Provedor equivalente, também com Pix e cartão. **Por que não:** Não foi comparado em profundidade nesta versão. O custo de trocar é baixo, porque a interface Gateway isola o provedor.

5 Consequências

Positivas

- A plataforma não armazena dados de cartão, reduzindo o risco e a exposição de dados pessoais.
- Resultados repetidos não duplicam cobrança nem reserva.
- O ambiente de testes e o GatewayFalso permitem testar o pagamento tardio sem cobrar ninguém.

Negativas

- A confirmação depende da disponibilidade, das taxas e dos prazos do provedor.
- As notificações exigem uma URL pública com HTTPS, e o teste local precisa de um túnel.
- O fluxo assíncrono é mais difícil de testar e exige simular o provedor.
- A devolução por API tem prazo máximo de 180 dias após a aprovação, e a política de cancelamento (RN-023) ainda não existe.

6 Requisitos afetados

- **Atende:** RNF-002, RNF-006 (a plataforma não coleta dados de cartão), RNF-007
- **Compromete:** RNF-004 (a confirmação depende de um serviço fora do nosso controle)

7 Riscos e gatilho de reavaliação

Reavaliar se o provedor falhar em entregar resultados de forma recorrente, se as taxas inviabilizarem o modelo ou se a política de cancelamento (RN-023) exigir devoluções parciais, que mudariam Gateway.estornar.

1 Identificação do ADR

ADR-006 · Definir a expiração da reserva temporária pelo relógio do servidor, com o prazo gravado na Reserva

Status: proposta | **Data:** 05/11/2026

2 Contexto

A RN-002 fixa a reserva temporária em 10 minutos, com contador visível, e determina que sair, fechar a tela ou repetir o pagamento não reinicia o prazo. A RN-004 estabelece que aprovação recebida no instante da expiração ou depois não confirma. No UC-09, o passo 3 cria a reserva "expira em 10 minutos" e a mensagem 10 informa que o tempo expirou.

O relógio do celular pode estar errado ou ser alterado, o aplicativo pode ser fechado a qualquer momento e o servidor pode reiniciar ou ser restaurado de uma cópia. O prazo precisa sobreviver a tudo isso e ser auditável (RN-020).

3 Decisão

Gravamos o instante de expiração na própria Reserva, calculado pelo relógio do servidor, e o usamos como única referência de prazo.

Reserva.criarTemporaria() grava expiraEm como criadaEm mais 10 minutos, e esse é o único lugar do sistema onde o prazo existe como regra. O aplicativo só exibe a contagem regressiva a partir de expiraEm e anuncia início, último minuto e expiração (RN-022). Reserva.confirmar(pagamento) compara pagamento.aprovacaoRecebidaEm com expiraEm e trata igual ou posterior como expirada. Reserva.expirar() é acionada por um processo periódico, e a transação que cria uma nova reserva expira antes as temporárias vencidas do mesmo período (ADR-004), de modo que o horário volta a ficar livre no instante do vencimento.

4 Alternativas consideradas

Temporizador em memória para cada reserva
O que era: O servidor ou o aplicativo manteria um contador por reserva. **Por que não:** O estado se perde ao reiniciar o servidor ou fechar o aplicativo, e o contador no cliente pode ser reiniciado ou adulterado.

Expiração por validade de cache
O que era: A reserva seria uma chave de cache com prazo de vida de 10 minutos. **Por que não:** O vencimento não é exato e não deixa registro persistente do estado "expirada" (RN-016, RN-020).

Confiar na hora informada pelo aplicativo
O que era: O aplicativo enviaria o horário de início da reserva e do pagamento. **Por que não:** O horário do dispositivo pode estar incorreto ou ser manipulado, o que permitiria estender o prazo.

5 Consequências

Positivas

- O prazo é o mesmo em qualquer aparelho e sobrevive a falhas e a restaurações de cópia.
- Se o cliente mudar o prazo, abrimos um arquivo: a classe Reserva.
- Cada expiração gera um evento rastreável.

Negativas

- Há um processo periódico a mais para manter e monitorar.
- O contador no aplicativo pode divergir do servidor em alguns segundos por causa da latência.
- Os relógios dos servidores precisam estar sincronizados.

6 Requisitos afetados

- **Atende:** RNF-004, RNF-007, RNF-008 (os prazos persistidos permitem reconciliar após restauração), RNF-009 (contador que anuncia só marcos, sem depender de cor)
- **Compromete:** Nenhum RNF numericamente; a diferença de alguns segundos entre contador e servidor é aceita, e vale sempre a decisão do servidor

7 Riscos e gatilho de reavaliação

Reavaliar se o prazo de 10 minutos mudar por política, ou se o volume de reservas exigir que a varredura de expiradas ocorra em intervalo menor.

1 Identificação do ADR

ADR-007 · Centralizar a autorização por perfil e vínculo de espaço antes de cada serviço

Status: proposta | **Data:** 05/11/2026

2 Contexto

O RNF-002 exige impedir 100% dos acessos não autorizados a reservas, agenda e financeiro de outra conta. A RN-005 limita nome e telefone do responsável a reservas confirmadas ou concluídas e proíbe expor CPF, senha, cartão e reservas de outros espaços. A RN-007 reserva ao proprietário vinculado a gestão do espaço, e a RN-009 restringe o financeiro às transações do próprio espaço.

Se cada tela ou cada rota verificar permissões por conta própria, o esquecimento de uma única verificação já expõe dados de outra pessoa, e o erro só aparece depois do vazamento.

3 Decisão

Toda requisição passa pela autenticação e por uma verificação central de autorização, que considera perfil, conta e espaço vinculado, antes de chegar a um serviço.

A verificação central é feita por guards do NestJS aplicados a todas as rotas, a partir do perfil e da conta do token (ADR-003). Os repositórios de agenda, reservas e financeiro exigem sempre o identificador do jogador ou do espaço autenticado como filtro obrigatório, sem consultas por identificador solto. A projeção dos dados do responsável pela reserva (RN-005) é decidida pelo estado da Reserva, no domínio. Os menus por perfil (RN-013) servem à usabilidade e não são tratados como proteção.

4 Alternativas consideradas

Verificar permissão manualmente em cada rota ou tela
O que era: Cada desenvolvedor incluiria a checagem no código de sua funcionalidade. **Por que não:** Depende de lembrança, é difícil de auditar e torna inviável demonstrar o critério de 100% do RNF-002.

Segurança por linha no banco como única barreira
O que era: O banco filtraria as linhas visíveis a cada conta. **Por que não:** Regras que dependem do estado da reserva (RN-005) são mais simples de testar no domínio, e a configuração por linha aumentaria a complexidade para a equipe nesta versão.

Apenas ocultar funções no aplicativo conforme o perfil
O que era: O aplicativo não exibiria o que o perfil não pode usar. **Por que não:** O cliente não é confiável: uma chamada direta à API contornaria a ocultação.

5 Consequências

Positivas

- Um ponto único concentra o teste do RNF-002: as 10 tentativas entre contas viram teste automatizado.
- Novas rotas herdam a verificação por padrão.
- A recusa de acesso pode ser registrada de forma uniforme (RN-020).

Negativas

- Toda consulta passa a exigir o contexto de identidade como parâmetro.
- Um erro na verificação central afeta todos os serviços de uma só vez.
- Registrar recusas aumenta o volume de eventos a reter por 180 dias.

6 Requisitos afetados

- **Atende:** RNF-002, RNF-006, RNF-007, RNF-003 (menus coerentes com o perfil)
- **Compromete:** RNF-001 (a verificação adicional em cada requisição tem custo pequeno, a medir na busca)

7 Riscos e gatilho de reavaliação

Reavaliar se surgirem novos perfis, como administrador de espaço público, se o bate-papo (RN-010) entrar no escopo, ou diante de qualquer falha nos testes do RNF-002.

1 Identificação do ADR

ADR-008 · Usar serviços gerenciados para hospedagem (Render), e-mail (Resend), fotos (Supabase Storage) e cópias de segurança próprias a cada 6 horas

Status: proposta | **Data:** 05/11/2026

2 Contexto

O RNF-004 exige disponibilidade mensal de 99,5%, e a RN-021 exige cópias criptografadas a cada 6 horas, retidas por 30 dias em ambiente separado, com restauração em até 2 horas e perda máxima de 6 horas, reconciliação antes de novas reservas e verificação mensal. O RF-007 exige fotos de espaços e quadras, e os RF-012 e RF-016 exigem e-mails de confirmação de conta e de recuperação de senha.

A equipe não tem pessoa dedicada à operação, e as cópias nativas de um banco gerenciado nem sempre cumprem a frequência e a retenção da RN-021.

3 Decisão

Usamos serviços gerenciados: Render para a API e para o job de cópias, Supabase para o banco e as fotos, e Resend para os e-mails, além de cópias de segurança próprias a cada 6 horas.

A API roda como serviço web no Render, em plano que a mantenha sempre ativa. O banco e o armazenamento de fotos ficam no Supabase, com fotos entregues por URLs assinadas, já que só o jogador autenticado consulta quadras (RN-001). Os e-mails saem pela API do Resend, com domínio verificado. A documentação do Supabase indica cópias diárias automáticas apenas em planos pagos (7 dias de retenção no plano Pro), então adicionamos um job agendado que executa pg_dump a cada 6 horas, criptografa o arquivo e o envia a um armazenamento de objetos em conta separada, apagando o que passa de 30 dias. A restauração é ensaiada todo mês, medindo o tempo e a reconciliação de agenda, pagamentos e eventos.

4 Alternativas consideradas

Servidor próprio (VPS com Docker)
O que era: A equipe instalaria e manteria API, banco e rotinas de cópia. **Por que não:** Exigiria operar e monitorar tudo, e atingir 99,5% e restauração em 2 horas dependeria de rotinas que a equipe não tem tempo de montar.

Nuvem de grande porte completa (banco, armazenamento e e-mail no mesmo provedor)
O que era: Um provedor único, com cópias contínuas nativas. **Por que não:** Cumpriria bem a RN-021, mas exige configuração de rede e permissões acima do que o semestre comporta.

Apenas as cópias nativas do banco gerenciado
O que era: Usar somente o recurso do plano do Supabase. **Por que não:** As cópias diárias não cumprem a frequência de 6 horas nem a retenção de 30 dias da RN-021, e ficam no mesmo ambiente.

5 Consequências

Positivas

- A equipe não opera servidores, o que libera tempo para as regras do negócio.
- A RN-021 é cumprida pelas cópias próprias, em ambiente separado e criptografadas.
- As fotos ficam protegidas por URL assinada, coerente com a RN-001.

Negativas

- Planos pagos têm custo mensal, e os planos gratuitos não oferecem cópias nem mantêm a API sempre ativa.
- A disponibilidade passa a depender de três fornecedores e de seus planos e limites.
- O job de cópia é código nosso, que precisa ser mantido e testado.
- A restauração de uma cópia lógica pode levar mais tempo que as cópias nativas, e a meta de 2 horas precisa ser medida.
- As chaves de criptografia das cópias precisam de guarda separada.

6 Requisitos afetados

- **Atende:** RNF-008, RNF-004 (parcialmente, pelo uso de hospedagem gerenciada)
- **Compromete:** RNF-004 (a disponibilidade depende dos fornecedores contratados)

7 Riscos e gatilho de reavaliação

Reavaliar se a verificação mensal não restaurar em até 2 horas, se algum fornecedor mudar planos ou limites, ou se a disponibilidade mensal ficar abaixo de 99,5%.